Universität Duisburg-Essen
Fakultät für Informatik
Prof. Dr.-Ing. Lucas Davi
Noah Kappert
Firas Khamis

UNIVERSITÄT DUISBURG ESSEN
Open-Minded

Übungsblatt 26.10.2026

Vorlesung „Secure Software Systems“
Wintersemester 2026/27

Theoretische Übung
Shellcode Analyse

In dieser Übung werden wir die Grundlagen zur Shellcode Analyse besprechen. Diese Übung dient der Vorbereitung für die praktischen Übungen zu Assembler und Laufzeitangriffen.

1 Shellcode Analyse (x86 32-bit)

Gegeben sei folgender Shellcode[1]:

```
1  31 c0                   xor    eax,eax
2  50                      push   eax
3  68 6e 2f 73 68          push   0x68732f6e
4  68 2f 2f 62 69          push   0x69622f2f
5  89 e3                   mov    ebx,esp
6  99                      cdq
7  52                      push   edx
8  53                      push   ebx
9  89 e1                   mov    ecx,esp
10 b0 0b                   mov    al,0xb
11 cd 80                   int    0x80
```

Die Instruktion `cdq` steht für *Convert word to doubleword* und schreibt das höchste Bit von `EAX` in alle Bits des Registers `EDX`. Zur Lösung der Aufgabe kann die Verwendung von Anhang A und B hilfreich sein.

i) Beschreiben Sie jeweils in einem Satz die Bedeutung jeder Instruktion.

ii) Fügen Sie in den unten abgebildeten Stack Frame die 5 Speicherwörter ein, die vom Shellcode auf dem Stack geschrieben werden.

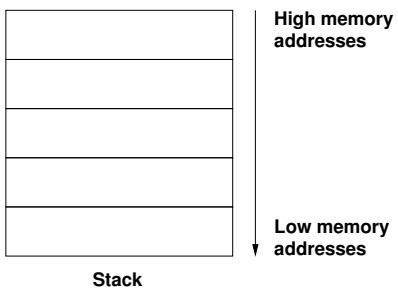


Abbildung 1: Stack Frame (Stack wächst nach unten)

[1] Quelle: `http://www.exploit-db.com/exploits/13444/`

iii) Welche Werte beinhalten die Register `EAX` bis `EDX` bevor die letzte Instruktion (`INT 0x80`) ausgeführt wird. Gebe Sie konkrete Werte für die Register `EAX` und `EDX` an. Die Register `EBX` und `ECX` werden Adressen beinhalten die erst zur Laufzeit des Programms berechnet werden. Geben Sie jedoch mithilfe von der Abbildung aus Teilaufgabe *(i)* an, auf welche Worte diese im Speicher zeigen werden.

- EAX:
- EBX:
- ECX:
- EDX:

iv) Was ist der Zweck der zweiten, dritten und vierten Instruktion in obigen Shellcode?

v) Welche Bedeutung hat der Hexwert `0xb` in Instruktion 10?

2 Shellcode Analyse 2 (x86 32-bit)

Gegeben ist folgender Shellcode. Analysieren und Reverse-engineeren Sie den Shellcode und erklären Sie dabei die Funktionalität.

```
1  0x00000000      e8 00000000      call 0x05   ; relative call +5
2  0x00000005      59               pop ecx
3  0x00000006      83 c1 1e         add ecx, 0x1e
4  0x00000009      b8 04000000      mov eax, 4
5  0x0000000e      bb 01000000      mov ebx, 1
6  0x00000013      ba 0d000000      mov edx, 0xd
7  0x00000018      cd 80            int 0x80
8  0x0000001a      b8 01000000      mov eax, 1
9  0x0000001f      31 db            xor ebx, ebx
10 0x00000021      cd 80            int 0x80
11 0x00000023      48               dec eax
12 0x00000024      65 6c            insb byte es:[edi], dx
13 0x00000026      6c               insb byte es:[edi], dx
14 0x00000027      6f               outsd dx, dword [esi]
15 0x00000028      20 57 6f         and byte [edi + 0x6f], dl
16 0x0000002b      72 6c            jb 0x99
17 0x0000002d      64 21 0a         and dword fs:[edx], ecx
```

Aufgabe 1.

i) Analysieren Sie die ersten zwei Instruktionen: Was genau macht eine `call` Instruktion? Welche Effekte hat die `call` Instruktion auf die Register und auf den Stack? Welchen Wert hat das Register `ECX` nach der zweiten Instruktion?

ii) Welche System Calls werden durch die zwei `int 0x80` Instruktionen ausgeführt (siehe Appendix A)? Welche Werte haben die Register jeweils bei den `int 0x80` Instruktionen?

Vor `int 0x80` an der Adresse `0x00000018`:

`EAX:` `EBX:` `ECX:` `EDX:`

Vor `int 0x80` an der Adresse `0x00000021`:

`EAX:` `EBX:`

iii) Erklären Sie die Instruktionen nach der zweiten `int 0x80` Instruktion: Was ist deren Bedeutung? Werden diese ausgeführt? Wo werden diese referenziert? (Tipp: Appendix B)

Anhang

A Linux System Calls für x86 32-bit

Tabelle 1 zeigt einen vereinfachten Ausschnitt aus der „System Call Table“ des Linux Kernels. Darin sollten sich alle wichtigen System Calls befinden, die Ihnen während der Übungen begegnen.

Tabelle 1: First 14 Linux Systems Calls on x86 32-bit

Number	Name	eax	ebx	ecx	edx	esi	edi
0	sys_restart_syscall	0x00	-	-	-	-	-
1	sys_exit	0x01	int error_code	-	-	-	-
2	sys_fork	0x02	struct pt_regs *	-	-	-	-
3	sys_read	0x03	unsigned int fd	char *buf	size_t count	-	-
4	sys_write	0x04	unsigned int fd	char *buf	size_t count	-	-
5	sys_open	0x05	char *filename	int flags	int mode	-	-
6	sys_close	0x06	unsigned int fd	-	-	-	-
7	sys_waitpid	0x07	pid_t pid	int *stat_addr	int options	-	-
8	sys_creat	0x08	char *pathname	int mode	-	-	-
9	sys_link	0x09	char *oldname	char *newname	-	-	-
10	sys_unlink	0x0a	char *pathname	-	-	-	-
11	sys_execve	0x0b	char* filename	char** argv	char** envp	struct pt_regs *	-
12	sys_chdir	0x0c	char *filename	-	-	-	-
13	sys_time	0x0d	time_t *tloc	-	-	-	-
14	sys_mknod	0x0e	char *filename	int mode	unsigned dev	-	-

B The ASCII Encoding

In C, strings are bytes that follow the ASCII encoding. Each numeric byte value corresponds to a character, some of which are printable, while others have special meaning, such as the null terminator or the newline character. Originally, ASCII was restricted to 7 bits, but was later extended to the full 8 bits/a single byte. In C, binary and printable data is often intermixed and this lack of distinction becomes important when analyzing or exploiting low-level vulnerabilities.
The full conversion table can be found in Figure 2.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	*[NULL]*	32	20	*[SPACE]*	64	40	@	96	60	`
1	1	*[START OF HEADING]*	33	21	!	65	41	A	97	61	a
2	2	*[START OF TEXT]*	34	22	"	66	42	B	98	62	b
3	3	*[END OF TEXT]*	35	23	#	67	43	C	99	63	c
4	4	*[END OF TRANSMISSION]*	36	24	$	68	44	D	100	64	d
5	5	*[ENQUIRY]*	37	25	%	69	45	E	101	65	e
6	6	*[ACKNOWLEDGE]*	38	26	&	70	46	F	102	66	f
7	7	*[BELL]*	39	27	'	71	47	G	103	67	g
8	8	*[BACKSPACE]*	40	28	(	72	48	H	104	68	h
9	9	*[HORIZONTAL TAB]*	41	29	)	73	49	I	105	69	i
10	A	*[LINE FEED]*	42	2A	*	74	4A	J	106	6A	j
11	B	*[VERTICAL TAB]*	43	2B	+	75	4B	K	107	6B	k
12	C	*[FORM FEED]*	44	2C	,	76	4C	L	108	6C	l
13	D	*[CARRIAGE RETURN]*	45	2D	-	77	4D	M	109	6D	m
14	E	*[SHIFT OUT]*	46	2E	.	78	4E	N	110	6E	n
15	F	*[SHIFT IN]*	47	2F	/	79	4F	O	111	6F	o
16	10	*[DATA LINK ESCAPE]*	48	30	0	80	50	P	112	70	p
17	11	*[DEVICE CONTROL 1]*	49	31	1	81	51	Q	113	71	q
18	12	*[DEVICE CONTROL 2]*	50	32	2	82	52	R	114	72	r
19	13	*[DEVICE CONTROL 3]*	51	33	3	83	53	S	115	73	s
20	14	*[DEVICE CONTROL 4]*	52	34	4	84	54	T	116	74	t
21	15	*[NEGATIVE ACKNOWLEDGE]*	53	35	5	85	55	U	117	75	u
22	16	*[SYNCHRONOUS IDLE]*	54	36	6	86	56	V	118	76	v
23	17	*[ENG OF TRANS. BLOCK]*	55	37	7	87	57	W	119	77	w
24	18	*[CANCEL]*	56	38	8	88	58	X	120	78	x
25	19	*[END OF MEDIUM]*	57	39	9	89	59	Y	121	79	y
26	1A	*[SUBSTITUTE]*	58	3A	:	90	5A	Z	122	7A	z
27	1B	*[ESCAPE]*	59	3B	;	91	5B	[	123	7B	{
28	1C	*[FILE SEPARATOR]*	60	3C	<	92	5C	\	124	7C	|
29	1D	*[GROUP SEPARATOR]*	61	3D	=	93	5D	]	125	7D	}
30	1E	*[RECORD SEPARATOR]*	62	3E	>	94	5E	^	126	7E	~
31	1F	*[UNIT SEPARATOR]*	63	3F	?	95	5F	_	127	7F	*[DEL]*

Abbildung 2: ASCII Table